BookVerse Step-by-Step Build Prompts

Local-first · ০ টাকা খরচ · ১৭টি প্রম্পট, একটার পর একটা

Astro + Preact | Express + Prisma + PostgreSQL | TanStack Query + Zustand | Tailwind + TypeScript | Fresh & Human colours

সংস্করণ ১.০ · ৫ অক্টোবর ২০২৬ · সঙ্গের ফাইল: README.md, PROJECT_FLOW.md, AGENTS.md, docs/FUTURE_PLAN.md

১. কীভাবে ব্যবহার করবেন

1. **Prompt 0** (পৃষ্ঠা ৬ থেকে) পুরোটা কপি করে রিপোর মূলে `AGENTS.md` নামে রাখুন (Claude Code হলে `CLAUDE.md`)। এটা একবারই।
2. এরপর **P1** থেকে শুরু করে একটা একটা প্রম্পট আপনার AI কোডিং টুলে পেস্ট করুন। একসাথে দুটো নয়।
3. যেখানে `[PASTE APPENDIX X]` লেখা, সেখানে এই PDF-এর শেষের Appendix X-এর লেখা কপি করে বসান।
4. AI-র রিপোর্ট পড়ুন, তারপর **ACCEPTANCE** নিজে চালিয়ে মিলিয়ে দেখুন। মিললে `git commit` করে পরেরটায় যান।
5. টাকা, লগইন, follow নিয়ম ও webhook কোডের diff নিজে পড়বেন। এখানেই ভুলের দাম সবচেয়ে বেশি।
6. AI যা অনিশ্চিত বলবে (`VERIFY-BEFORE-LAUNCH`), তা `docs/FUTURE_PLAN.md`-এর শেষে টুকে রাখুন।

পুরো পথের মানচিত্র

ধাপ	প্রম্পট	কী হয়	শেষে আপনার হাতে
A · Setup	1–2	প্রজেক্ট দাঁড়ায়, ডিজাইন ও রঙ ঠিক হয়	docker + pnpm dev চলে, /design পেজ
B · Data + Auth	3–4	ডাটাবেস, লগইন, রোল, follow নিয়ম	নিরাপদ ভিত্তি, টেস্ট-সহ
C · Commerce	5–9	বই খোঁজা → কেনা → টাকা → ডেলিভারি → payout	MVP দোকান (লোকালে পূর্ণ)
D · Retention	10–12	review, alert, feed, shelf, points	মানুষ ফিরে আসার কারণ
E · Differentiators	13–16	used-book, mood, publisher tools, events	Rokomari-র সাথে পার্থক্য
F · Hardening	17	নিরাপত্তা, ব্যাকআপ, লোড টেস্ট	হোস্টিংয়ের আগে প্রস্তুতি

লোকাল মোড: কোনো domain, hosting বা paid সার্ভিস নেই। Postgres, Redis ও ইমেইল ইনবক্স (Mailpit) Docker-এ চলে। পেমেন্ট, courier, ছবি-সেভ, SMS ও ইমেইল প্রতিটি একটি "provider"-এর পেছনে, তাই পরে শুধু `.env` বদলালেই আসল সার্ভিস চালু হয়। পরের আপগ্রেডের তালিকা `docs/FUTURE_PLAN.md`-এ।

২. রঙ: "ফ্রেশ ও মানবিক" ভাইব

আপনার প্রশ্নের উত্তর: হ্যাঁ, রঙের মনস্তত্ত্ব আগের প্ল্যানেই ছিল: নীল = বিশ্বাস, কমলা = অ্যাকশন, সবুজ = প্রবৃদ্ধি/ফ্রেশ, লাল = জরুরি, হলুদ = আনন্দ, বেগুনি = প্রিমিয়াম, সাদা ফাঁকা জায়গা = শান্ত। কিন্তু আগের টোকেনে পেজের ব্যাকগ্রাউন্ড ছিল ঠান্ডা ধূসর-নীল (#F8FAFC), তাই সাইটটা "অফিস ড্যাশবোর্ড" মনে হতে পারত, "ফ্রেশ ও মানবিক" নয়। তাই এই সংস্করণে রঙ বদলানো হয়েছে (**Palette v2**) এবং `AGENTS.md`-ও আপডেট করা হয়েছে।

কী বদলেছে

- **উষ্ণ ক্রিম ব্যাকগ্রাউন্ড** (#FBF7F0): বইয়ের কাগজের মতো নরম। সাদা কার্ড তার উপর ভেসে থাকে।
- **Leaf সবুজ** ফ্রেশ অনুভূতির জন্য: স্টকে আছে, পড়ার অগ্রগতি, ছোট আইকন ও নরম পটভূমি।
- **নরম pastel mood chip:** ১০টি মেজাজের প্রতিটির নিজস্ব রঙ। মানুষ নিজের অনুভূতির রঙ বেছে নেয়, এটাই "মানবিক" স্পর্শ।
- **কমলা বাটনে গাঢ় লেখা**: আগে সাদা লেখা দিলে পড়া কঠিন হতো (contrast ফেল)।
- অনুপাত: ~৭০% ক্রিম/সাদা, ~১৫% নীল, ~৮% সবুজ, ~৫% কমলা। লাল শুধু জরুরি জিনিসে।

রঙের তালিকা

paper #FFFFFF পেজের কার্ডফর্ম	**cream** #FBF7F0 ডিফল্ট পেজ ব্যাকগ্রাউন্ড (বইয়ের কাগজের উষ্ণতা)	**sand** #F3EDE2 বিকল্প সেকশন
ink-900 #1F2A37 মূল লেখা	**ink-500** #5B6472 দ্বিতীয় লেখা	**line** #EAE3D6 বর্ডার
brand-600 #2563EB বিশ্বাস: নেভ, লিংক, প্রধান বাটন	**leaf-600** #16A34A ফ্রেশ/প্রবৃদ্ধি: স্টকে আছে, অগ্রগতি	**cta-500** #F97316 একটাই প্রধান অ্যাকশন (লেখা গাঢ়)
sun-200 #FDE68A অফার, ফ্রি ডেলিভারি	**rose-600** #E11D48 শুধু লাইক/হার্ট	**urgent-600** #DC2626 জরুরি: ফ্ল্যাশ সেল, ভুল
premium-600 #7C3AED ভেরিফায়েড, VIP	**brand-100** #DBEAFE নরম নীল পটভূমি	**leaf-100** #DCFCE7 নরম সবুজ পটভূমি

মেজাজ অনুযায়ী রঙিন chip (mood picker)

Happy · Calm · Sad · Motivated · Curious · Romantic · Lonely · Stressed · Adventurous · Focused

পড়ার যোগ্যতা যাচাই (WCAG AA ≥ ৪.৫)

জোড়া	নমুনা	অনুপাত	ফল
Body text ink-900 on cream	Aa বই	13.61	PASS
Secondary text ink-500 on cream	Aa বই	5.60	PASS
Secondary text ink-500 on white	Aa বই	5.98	PASS
Link / brand text brand-700 on cream	Aa বই	6.28	PASS
Button text white on brand-600	Aa বই	5.17	PASS
CTA text ink-900 on cta-500	Aa বই	5.19	PASS
CTA text white on cta-700	Aa বই	5.18	PASS
Success text leaf-700 on leaf-50	Aa বই	4.79	PASS
Success button white on leaf-700	Aa বই	5.02	PASS
Urgent text white on urgent-600	Aa বই	4.83	PASS
Premium text white on premium-600	Aa বই	5.70	PASS
Offer tag sun-800 on sun-100	Aa বই	6.37	PASS
Heart/like rose-600 on white	Aa বই	4.70	PASS
Mood Happy	Aa বই	6.37	PASS
Mood Calm	Aa বই	6.49	PASS
Mood Sad	Aa বই	8.49	PASS
Mood Motivated	Aa বই	6.38	PASS
Mood Curious	Aa বই	6.73	PASS
Mood Romantic	Aa বই	6.68	PASS
Mood Lonely	Aa বই	7.57	PASS
Mood Stressed	Aa বই	6.59	PASS
Mood Adventurous	Aa বই	6.52	PASS
Mood Focused	Aa বই	11.87	PASS

সৎ নোট: রঙ কেমন "ফ্রেশ" লাগে তা ব্যক্তি ও সংস্কৃতিভেদে বদলায়, কোনো ফর্মুলা নেই। Prompt 2 শেষ হলে `/design` পেজ ৫–১০ জন আসল পাঠককে দেখান। জিজ্ঞেস করুন: "কেমন লাগছে? ফ্রেশ, শান্ত, বিশ্বাসযোগ্য?" তারপর শুধু টোকেন ফাইলে রঙ বদলান (কোড বদলাতে হবে না)।

৩. Prompt 0 — AGENTS.md (একবার পেস্ট)

নিচের পুরো ব্লকটি রিপোর মূলে `AGENTS.md` (বা `CLAUDE.md`) নামে সেভ করুন।

```
PROJECT: BookVerse (working name) - social-commerce platform for books in Bangladesh.
Currency BDT. Languages: Bangla (primary) + English. Mobile-first.
Business: marketplace (publishers, writers, vendors, users selling used books) + social reading
community. Launch scale: 10,000 active users, peak ~1,000 concurrent. Long-term GMV goal 15 crore BDT/yr.

CURRENT MODE: LOCAL-FIRST, ZERO COST. No domain, no hosting, no paid service.
Everything runs on the developer machine via docker-compose (postgres, redis, mailpit).
Every external service sits behind a provider interface in apps/api/src/providers/<name>/ with a
local/mock implementation (default) and a real implementation (selected by env, see README).
Do NOT add Cloudflare, Vercel or Render adapters now. Use @astrojs/node. Future upgrades are listed
in docs/FUTURE_PLAN.md; do not implement them unless asked.

FIXED STACK (do not substitute without asking):
- Monorepo (pnpm workspaces): apps/web, apps/api, packages/shared
- apps/web: Astro (SSR, @astrojs/node) + Preact (compat mode) + Tailwind CSS + TypeScript strict
  + TanStack Query (via preact/compat) + Zustand + GSAP (only small, purposeful animation)
- apps/api: Node + Express + TypeScript strict + Zod + Prisma + PostgreSQL + Redis + BullMQ
- Auth: Better Auth (DB sessions, role field). Phone OTP via SmsProvider (console in local mode).
- Storage: StorageProvider (local disk now, R2 later). Email: EmailProvider (SMTP→Mailpit now).
- Payments: PaymentProvider (mock / SSLCommerz sandbox now). Courier: CourierProvider (mock now,
  Steadfast adapter written but off). Search: pg_trgm + simple FTS behind SearchService.
- Tests: Vitest (+ Supertest for API), Playwright for a few critical e2e flows.

GOLDEN RULES (violating any of these is a bug):
1. The server is the source of truth. Price, stock, coupon, commission, delivery fee, permissions and
   payment status are ALWAYS computed/verified server-side. Never trust client values.
2. Money: Prisma Decimal(12,2), integer-paisa arithmetic, round-half-up. LedgerEntry amount/kind/owner
   are immutable (only status/availableAt change, by the escrow service); corrections are new ADJUSTMENT
   entries. Every money write is inside a DB transaction and has an idempotencyKey. Balances are SUMs.
   Points follow the same pattern (PointLedger; only awardPoints() touches pointsBalance).
3. Authorization lives in the service layer via policy functions (e.g. canFollow). Routes also use
   requireAuth/requireRole plus ownership checks. UI hiding is never security.
4. Every input is validated with a Zod schema from packages/shared.
5. Layering in api: route → controller → service → repository. No DB access in controllers.
   No business logic in Astro pages or Preact components.
6. Lists use cursor pagination (createdAt + id). Never OFFSET for feeds. Prisma select only needed
   fields. Add composite indexes for real query paths and check with EXPLAIN.
7. Astro islands do not share Preact Context. Shared state = module singletons
   (src/lib/queryClient.ts, src/stores/*.ts). Hydrate SSR data via initialData.
8. Webhooks (payment, courier): raw body, verify per the provider's CURRENT official docs, idempotent
   (PaymentEvent unique provider+eventId), log, respond fast, heavy work in BullMQ. Never trust browser
   redirects for payment success. Locally, provide simulator scripts (pnpm sim:ipn, pnpm sim:courier).
9. Business values (commission, escrow days, claim window, COD limits, point rules) live in the DB
   (SystemSetting, CommissionRule). Every admin change writes AdminAuditLog.
10. API envelope: { success, data, meta } and { success:false, code, message, errors }.
    Never leak stack traces.
11. Security baseline: Helmet, strict CORS (credentials, allow http://localhost:4321 in local mode),
    Redis rate limits on auth/otp/follow/post/search/checkout, Origin check on mutations, upload MIME +
    size checks, no secrets in git, .env.example maintained.
12. UX baseline: skeleton loading, empty state, error state with retry. 360px mobile baseline.
    WCAG 2.2 AA, keyboard accessible, prefers-reduced-motion respected.
13. External APIs: read the current official docs before coding. If you cannot verify a behaviour,
    say so, isolate it behind an adapter and add it to VERIFY-BEFORE-LAUNCH in docs/FUTURE_PLAN.md.
    Never invent endpoints or fields.
14. Prefer free/open-source. Anything paid must be marked PAID DEPENDENCY.
15. Feature flags (FEATURE_*) gate optional modules: routes not registered and UI hidden when off.
16. No dead code, no TODO placeholders in shipped paths, no `any` (except documented 3rd-party edge).

WORKFLOW FOR EVERY TASK:
1) Restate scope and list the files you will create/change.  2) Note risks/assumptions.
3) Implement in small steps.  4) Write tests (money, auth, policy and webhook logic are mandatory).
5) Run lint + typecheck + tests and fix failures.  6) Report: what is done, what is NOT done,
   how I can verify it manually, deviations from the spec and why.
7) Update PROJECT_FLOW.md checkbox suggestions and README/docs if commands or env keys changed.
```

```
Stop after the task. Do not start the next prompt on your own. Never silently skip a requirement.

DESIGN TOKENS - "Fresh & Human" palette v2 (only via Tailwind theme, never raw hex in components):
paper #FFFFFF | cream #FBF7F0 (default page bg) | sand #F3EDE2 | line #EAE3D6
ink-900 #1F2A37 | ink-700 #374151 | ink-500 #5B6472 | ink-300 #9AA3B0 (decor only)
brand-600 #2563EB (hover/links brand-700 #1D4ED8) | leaf-600 #16A34A, leaf-700 #15803D (text/buttons), leaf-50
#F0FDF4
cta-500 #F97316 (ink-900 text on it; white text only on cta-700 #C2410C) | sun-100 #FEF3C7 / sun-800 #92400E
rose-600 #E11D48 (like only) | urgent-600 #DC2626 (flash sale, out of stock, errors only) | premium-600 #7C3AED
Mood chips (bg/text): Happy #FEF3C7/#92400E, Calm #DCFCE7/#166534, Sad #DBEAFE/#1E3A8A,
Motivated #FFEDD5/#9A3412, Curious #CCFBF1/#115E59, Romantic #FFE4E6/#9F1239, Lonely #EDE9FE/#5B21B6,
Stressed #E0F2FE/#075985, Adventurous #ECFCCB/#3F6212, Focused #E2E8F0/#1E293B.
Proportion ~70% cream/white, ~15% blue, ~8% leaf green, ~5% orange CTA. Max 2 orange CTAs per screen.
Radius 12px, soft warm shadow. Fonts: Hind Siliguri (Bangla) + Inter (English), self-hosted, swap.

OUT OF SCOPE (do NOT build unless asked): own Rider app, video streaming, native mobile app,
subscription billing, ML recommendations, smooth-scroll libraries, 3D hero, any hosting setup.
```

৪. ধাপে ধাপে প্রম্পট P1–P17

প্রতিটি প্রম্পট ইংরেজিতে (AI টুল সবচেয়ে ভালো বোঝে), ব্যাখ্যা বাংলায়। বাক্স থেকে হুবহু কপি করুন।

P1 Monorepo, tooling, Docker A · Setup

প্রজেক্টের প্রথম ধাপ। খালি রিপোতে AGENTS.md রেখে এটি দিন।

```
TASK: Initialize the BookVerse monorepo. LOCAL-FIRST, zero cost. Read AGENTS.md first.

1. pnpm workspace: apps/web (Astro output 'server', @astrojs/node standalone, @astrojs/preact
   with compat:true, Tailwind), apps/api (Express + TS, tsx for dev, tsup for build),
   packages/shared (Zod schemas + types, built with tsup, imported by both apps).
2. TypeScript strict everywhere, shared tsconfig base, path aliases. Vitest in every package.
3. ESLint + Prettier + Husky + lint-staged.
4. docker-compose.yml: postgres 16, redis 7 (appendonly yes), mailpit (SMTP 1025, UI 8025),
   named volumes and healthchecks.
5. .env.example (used by both apps) with:
   DATABASE_URL, DIRECT_URL, REDIS_URL, WEB_ORIGIN=http://localhost:4321,
   API_ORIGIN=http://localhost:4000, PUBLIC_API_URL, API_INTERNAL_URL,
   BETTER_AUTH_SECRET, BETTER_AUTH_URL,
   STORAGE_PROVIDER=local, EMAIL_PROVIDER=smtp, SMS_PROVIDER=console, PAYMENT_PROVIDER=mock,
   COURIER_PROVIDER=mock, LLM_PROVIDER=off, SEARCH_PROVIDER=pg,
   SSLCOMMERZ_STORE_ID / SSLCOMMERZ_STORE_PASSWORD (sandbox), STEADFAST_* and R2_* (empty),
   FEATURE_RIDER=false, FEATURE_LIVE_EVENTS=false, FEATURE_SPONSORED=false,
   FEATURE_MOOD_LLM=false, FEATURE_USED_BOOKS=false.
   Validate env with Zod at startup in both apps and fail fast with a clear message.
6. Root scripts: dev, build, lint, typecheck, test, db:migrate, db:seed, db:studio, backup,
   reconcile, sim:ipn, sim:courier (the last four may print "not implemented yet" for now).
7. API foundations: GET /health (checks db + redis), request id, pino logger, central error
   handler, response envelope helpers, AppError with typed codes, and a provider registry
   (apps/api/src/providers/index.ts) that reads *_PROVIDER from env.
8. GitHub Actions: install, lint, typecheck, test on every PR.
9. Keep README.md, PROJECT_FLOW.md and docs/FUTURE_PLAN.md accurate (commands, ports).
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): `docker compose up -d && pnpm dev` চালালে web (4321) ও api (4000) ওঠে। `/health` ok। shared প্যাকেজ দুই অ্যাপেই import হয়। lint, typecheck, test পাস।

P2 Design system ও Astro shell (Fresh palette) A · Setup

রঙ ও UI-র ভিত্তি। এখানেই "ফ্রেশ ভাইব" তৈরি হয়।

```
TASK: Build the design system and Astro application shell.

TOKENS (Tailwind theme + CSS variables; never raw hex inside components):
  paper #FFFFFF | cream #FBF7F0 (default page background, warm book-paper feel)
  sand #F3EDE2 (alt sections) | line #EAE3D6 (borders)
  ink-900 #1F2A37 (text) | ink-700 #374151 | ink-500 #5B6472 (secondary text) | ink-300 #9AA3B0 (decor only)
  brand-600 #2563EB (primary, trust) | brand-700 #1D4ED8 (hover, links) | brand-100 #DBEAFE | brand-50 #EFF6FF
  leaf-600 #16A34A (growth/fresh fills, icons) | leaf-700 #15803D (success buttons/text) | leaf-100 #DCFCE7 |
leaf-50 #F0FDF4
  cta-500 #F97316 (primary action; text on it is ink-900) | cta-600 #EA580C | cta-700 #C2410C (white text) |
cta-50 #FFF7ED
  sun-100 #FEF3C7 | sun-200 #FDE68A | sun-800 #92400E (offers, free-shipping, badges)
  rose-600 #E11D48 (like/heart only)
  urgent-600 #DC2626 | urgent-50 #FEF2F2 (flash sale, out of stock, errors ONLY)
  premium-600 #7C3AED | premium-50 #F5F3FF (verified, VIP)
MOOD CHIPS (background / text): Happy #FEF3C7/#92400E, Calm #DCFCE7/#166534,
  Sad #DBEAFE/#1E3A8A, Motivated #FFEDD5/#9A3412, Curious #CCFBF1/#115E59,
  Romantic #FFE4E6/#9F1239, Lonely #EDE9FE/#5B21B6, Stressed #E0F2FE/#075985,
  Adventurous #ECFCCB/#3F6212, Focused #E2E8F0/#1E293B.
PROPORTION: ~70% cream/white, ~15% brand blue, ~8% leaf green, ~5% orange CTA, rest functional.
Max 2 orange CTAs per screen. Radius 12px. Soft warm shadow rgba(31,42,55,0.08).
```

```
Fonts: Hind Siliguri (Bangla) + Inter (English), self-hosted via @fontsource, subset, preload,
font-display: swap.

BUILD:
- layouts/Base.astro: SEO head (title, description, canonical, OpenGraph), header with search box,
  nav, cart-count island; mobile bottom nav (Home, Discover, Community, Cart, Profile); footer.
- src/lib/queryClient.ts, src/lib/apiClient.ts (fetch with credentials, typed envelope, ApiError),
  src/islands/Island.tsx wrapper, src/stores/cart.ts (Zustand, persist) and ui.ts.
- UI primitives in Preact, no external UI library: Button, Input, Textarea, Select, Modal (native
  <dialog>), Drawer, Tabs, Badge, Avatar, Card, Skeleton, Toast, EmptyState, ErrorState,
  Price (BDT, optional Bangla digits), Rating, BookCard, MoodChip, SpoilerBlur (CSS blur + reveal button).
- Motion helper (GSAP): gsap.matchMedia() with prefers-reduced-motion, reveal-on-scroll
  (y 24 → 0, opacity, 0.4s, once), count-up, subtle hover lift. Nothing else. Always cleanup.
- Pages: / (placeholder sections) and /design (dev-only style guide: every component and state,
  every token swatch, the 10 mood chips).
REPORT a contrast table for every text/background pair used.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): মোবাইল Lighthouse ৯০+। প্রতিটি টেক্সট জোড়া WCAG AA পাস (রিপোর্টে টেবিল)। modal/drawer/tabs কিবোর্ডে চলে। ফন্টে layout shift নেই। `/design` পেজে রঙ দেখে আপনার "ফ্রেশ" লাগছে কিনা নিজে যাচাই করুন।

P3 ডাটাবেস: schema, migration, seed

B · Data + Auth

এখানে Appendix A (Prisma schema) পেস্ট করতে হবে।

```
TASK: Database layer. LOCAL-FIRST: Postgres from docker-compose.

CONTEXT - core Prisma schema:
[PASTE APPENDIX A HERE]

1. Extend it into the complete schema with explicit relations and onDelete: Better Auth tables
   (generate with its Prisma adapter and merge with User), OrderStatusHistory, Claim (orderId
   unique, reason, images[], status, resolution), Block, Review (isVerifiedBuyer set only by
   server), BookshelfItem, ReadingGoal, Wishlist (priceAtAdd), Notification +
   NotificationPreference, Post (type, hasSpoiler, isSponsored, campaignId, likeCount,
   commentCount), Comment, Reaction (LIKE|DISLIKE, one per user per target), Report,
   ReprintRequest (unique bookId+userId), BookClub, ClubMember, Campaign, WriterProfile,
   PublisherProfile, VendorProfile, RoleApplication. Add the missing back-relations so that
   `prisma validate` passes.
2. Migration enabling pg_trgm and unaccent; GIN trigram indexes on Book.title/titleBn/author;
   generated tsvector column with GIN for descriptions.
3. DATABASE_URL and DIRECT_URL both point to local Postgres for now. Check the CURRENT Prisma
   docs for the datasource/config syntax of the installed version.
4. Seed: 12 genres, 200 Bangla+English books with moodTags, 5 publishers, 10 writers, 3 vendors,
   50 users, sample posts/reviews, CommissionRule rows and SystemSetting rows from Appendix D
   and Appendix C.
5. packages/shared money helpers: toPaisa, fromPaisa, splitCommission(lineTotal, bps) with unit
   tests (1,000 random cases: sellerAmount + commission === lineTotal exactly).
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): `prisma validate`, migrate ও seed খালি DB-তে পাস। title সার্চ ও "buyer-এর অর্ডার" কোয়েরিতে EXPLAIN-এ index ব্যবহার হয়। money টেস্ট ১,০০০ কেসে পাস।

P4 Auth, RBAC, Follow policy

B · Data + Auth

Appendix B (Follow policy) পেস্ট করতে হবে। লোকালে OTP টার্মিনালে ও ইমেইল Mailpit-এ আসবে।

```
TASK: Authentication, roles and the follow policy. LOCAL-FIRST.

1. Better Auth in apps/api: cookie sessions (HttpOnly, SameSite=Lax; Secure only when not
   localhost), email + password, email verification and password reset (mail goes to Mailpit
   via EmailProvider=smtp), phone OTP through SmsProvider (console provider prints the OTP in
   the terminal; define the interface so a real BD SMS provider can be added later).
```

```
2. GET /me. Astro middleware protects private routes by calling /me; redirect to /login.
3. Middleware requireAuth, requireRole(...roles), requireOwnership(resolver). Role application
   API: POST /role-applications (WRITER|PUBLISHER|VENDOR with documents); admin approve/reject
   endpoints. No endpoint may ever let a user set their own role.
4. Follow module with canFollow(actor, target, ctx) exactly per Appendix B:
   [PASTE APPENDIX B HERE]
   POST/DELETE /users/:id/follow; check + insert in ONE transaction; unfollow removes the
   writer's follow-back; block removes both follows; Redis rate limit 50/hour.
5. Redis rate limits: login 5/min/IP, OTP 3 per 10 min per phone, signup, follow, post creation.
6. Tests: exhaustive actor x target role table test for canFollow; parallel-request race test;
   privilege-escalation and IDOR tests (user A cannot read/modify user B resources).
7. Web pages (islands + Zod forms): login, register, verify email/phone, forgot/reset password,
   each with loading, error and success states.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): policy টেস্ট সবুজ। PUBLISHER→USER follow `curl` দিয়েও 403। USER অ্যাডমিন endpoint ডাকতে পারে না। web (4321) ও api (4000) এর মধ্যে session চলে। Mailpit-এ verification মেইল আসে।

P5 ক্যাটালগ, সার্চ বইয়ের পেজ C · Commerce

লোকালে ছবি ডিস্কে সেভ হয় (StorageProvider=local)।

```
TASK: Catalog, search and the book page. LOCAL-FIRST.

1. API: GET /books (filters genre, price range, condition, language, type NEW|USED, sort; cursor
   pagination), GET /books/:slug, GET /search?q= and GET /search/suggest?q= (books, authors,
   publishers). Ranking: exact/prefix > pg_trgm similarity > FTS. Bangla and English queries,
   Bangla/English digit normalisation, light typo tolerance. Put it behind a SearchService
   interface so Meilisearch can replace it later. Cache popular queries in Redis for 60s.
2. Publisher/vendor listing CRUD (ownership enforced): create book + listing; image upload through
   StorageProvider=local into ./uploads (served by the API with correct headers), MIME and size
   validation, stock management.
3. Look Inside: publishers upload 5-10 preview page images; requires a copyright-permission
   checkbox; admin approval before public; served as optimised images only.
4. Astro pages (SSR + Cache-Control for future CDN): book detail with JSON-LD (Book/Product +
   BreadcrumbList) and OpenGraph, all listings (new + used) of the book, author and publisher
   pages, category pages, search page with a filter island and 250ms debounced autocomplete.
   Wishlist and add-to-cart are small islands.
5. A benchmark script on 100k seeded books reporting p50/p95 for search.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): সার্চ p95 < 100ms (১০০k বইয়ে; না হলে আসল সংখ্যা রিপোর্ট করুন)। বাংলা কোয়েরি ও টাইপো-তে ঠিকঠাক ফল। বইয়ের পেজে Lighthouse SEO ৯৫+।

P6 কার্ট, চেকআউট, অর্ডার C · Commerce

Appendix C ও D-র নিয়ম ব্যবহার হবে। শুধু COD দিয়ে টেস্ট করুন।

```
TASK: Cart quote, checkout and order lifecycle. Money rules are in Appendix C and D.

1. Client cart (Zustand) is only a convenience. POST /checkout/quote re-reads every listing and
   returns authoritative prices, stock status, per-seller grouping, delivery charge (config table
   by district/weight), coupon effect and totals.
2. POST /checkout (requires Idempotency-Key header): in ONE transaction create a Checkout and
   one Order per seller; atomically reserve stock (UPDATE ... WHERE stock - reservedQty ≥ qty,
   fail cleanly on conflict); resolve the commission rule per item (Appendix D) and snapshot
   commissionBps, commissionAmount, sellerAmount; write OrderStatusHistory.
3. Reservation expiry: unpaid prepaid checkouts release stock after 15 minutes via a BullMQ
   delayed job.
4. COD guards: phone verified, new-user cap, trustScore threshold, COD disabled after 3 RTOs;
   otherwise return a typed error telling the UI to offer prepaid.
5. Order state machine with an allowed-transitions table; every transition writes
   OrderStatusHistory and is the ONLY way status changes. Cancel before PACKED allowed and
```

```
   releases stock.
6. Web: cart drawer, checkout (address, delivery, coupon, payment method, confirmation), order
   list and order detail. Each step has loading, error and retry states.
TESTS: concurrent checkout of the last copy (exactly one wins); price tampering ignored;
duplicate Idempotency-Key returns the same result; invalid state transitions rejected.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): টেস্ট সবুজ। হাতে চালিয়ে: কার্টে দিন → quote → COD অর্ডার → অ্যাডমিন অর্ডার দেখে।

P7 পেমেন্ট, ledger, escrow, claim

C · Commerce

Appendix C পেস্ট করতে হবে। লোকালে IPN আসে না, তাই সিমুলেটর বানানো হবে।

```
TASK: Payments and the money flow. LOCAL-FIRST.

CONTEXT - money flow, settings and rules:
[PASTE APPENDIX C HERE]

1. PaymentProvider interface with MockProvider (default) and SslCommerzProvider (sandbox). Read
   the CURRENT SSLCommerz docs first; do not assume signature details. Verify status, tran_id,
   amount and currency against our DB via the Validation API before marking paid.
2. IPN endpoint: raw body, respond fast, process inside a DB transaction with PaymentEvent
   (provider+eventId unique) for idempotency. On success create ledger entries per OrderItem:
   seller SALE (PENDING), platform COMMISSION, PAYMENT_FEE. idempotencyKey like sale:{orderItemId}.
3. Local simulator: `pnpm sim:ipn <tranId>` posts a sandbox-shaped IPN to the local API (the real
   Validation API check still runs against the sandbox where possible, otherwise through the
   mock). Document how to test a real sandbox IPN with a temporary free tunnel.
4. COD: no ledger on order creation; ledger entries are created when the courier remit is
   confirmed (Prompt 8 calls this service); RTO writes RTO_COST and lowers buyer trustScore.
5. Escrow service: on DELIVERED set availableAt = deliveredAt + escrow_days; an hourly BullMQ job
   moves PENDING → AVAILABLE when due and the order has no open claim. Support a TEST CLOCK
   (env-controlled "now") so you can fast-forward days locally. Only status/availableAt may change.
6. Claims: buyer opens a claim within claim_window_hours with images; order becomes DISPUTED and
   entries freeze; admin resolves (refund writes negative REFUND entries reversing seller and
   commission, or reject).
7. Payouts: weekly batch endpoint for admin, minimum amount, status flow PENDING → PAID with a
   reference; marks entries PAID.
8. `pnpm reconcile` job comparing ledger sums with order subtotals; also runs daily via BullMQ.
TESTS (mandatory): duplicate IPN is a no-op; wrong amount rejected; refund reverses exactly; claim
freezes release; ledger balance equals expected after a full lifecycle; concurrent IPN and cancel.
List everything that depends on live-account behaviour as VERIFY-BEFORE-LAUNCH.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): সিমুলেটর দিয়ে: পেমেন্ট → confirmed → (সিমুলেটেড ডেলিভারি) → test clock এগিয়ে escrow release → payout রেকর্ড। reconcile মেলে।

P8 Courier (mock + Steadfast adapter)

C · Commerce

লোকালে mock courier চলে। Steadfast adapter লেখা থাকবে, চালু হবে পরে।

```
TASK: Courier integration. LOCAL-FIRST.

1. CourierProvider interface: createShipment, trackShipment, cancelShipment, parseWebhook (verify).
   MockCourierProvider (default) and SteadfastProvider (written but off). Read the CURRENT
   Steadfast merchant API docs before writing it; do not guess endpoints, auth headers or webhook
   verification. Pathao/RedX: interface stubs only.
2. When a seller marks an order PACKED, a BullMQ job creates the shipment with the COD amount (0 for
   prepaid), stores consignmentId/trackingCode, retries with backoff on errors, and the admin has a
   manual-retry button.
3. Webhook endpoint: verify per provider docs, idempotent, store the raw payload, map courier
   statuses to our order states via a table. DELIVERED → order DELIVERED + escrow start;
   RETURNED → RTO flow (restock, RTO_COST, trustScore). COD remit confirmation → call the COD
   ledger service from Prompt 7.
4. `pnpm sim:courier <consignmentId> <status>` posts simulated webhooks (valid, invalid signature,
```

```
   duplicate, out-of-order) to the local API.
5. A polling fallback job (every 30 minutes) for shipments whose webhook never arrived.
6. Buyer tracking timeline on the order page.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): সিমুলেটেড webhook (বৈধ, ভুল signature, ডুপ্লিকেট, উল্টো ক্রম) সঠিক আচরণ করে। প্রতিটি provider-অনুমান VERIFY-BEFORE-LAUNCH তালিকায় আছে।

P9 Seller ও Admin ড্যাশবোর্ড

C · Commerce

এটি শেষ হলে MVP দোকান পূর্ণ।

```
TASK: Dashboards (client:only islands behind role guards).

SELLER (publisher, vendor, writer): overview (sales, pending/available balance), listings and
stock, orders to pack, shipments, earnings with a per-order commission breakdown, payout history,
claims.
ADMIN: user and role-application approvals, seller verification, orders, claims queue with
evidence, payout approval, commission rules and system settings editor (every change writes
AdminAuditLog with before/after), margin-guard warnings, basic KPIs (GMV, orders, AOV, RTO rate,
claim rate, take rate).
All data via paginated endpoints, ownership enforced server-side (a seller can never see another
seller's ledger). Tables have skeletons, empty and error states; mobile-friendly.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): প্রতিটি ড্যাশবোর্ড endpoint-এ IDOR টেস্ট। seed করা পরিস্থিতিতে KPI ledger ও অর্ডার ডেটার সাথে মেলে। অ্যাডমিন পরিবর্তন audit log-এ আসে।

P10 Review, Wishlist, Notification, Worker

D · Retention

ইমেইল Mailpit-এ আসে; আসল ইমেইল পরে।

```
TASK: Trust and retention basics. LOCAL-FIRST.

1. Reviews: one per user per book. isVerifiedBuyer is set ONLY server-side when the user has a
   DELIVERED order containing that book. Verified reviews are shown separately from social posts.
   Helpful votes, report button, hasSpoiler flag with blur UI. Photos allowed; audio later.
2. Wishlist with priceAtAdd; price-drop and back-in-stock alerts via BullMQ jobs triggered from
   the listing update service (no polling of every row).
3. Notification system: domain events → BullMQ → check NotificationPreference → in-app row +
   email through EmailProvider (smtp→Mailpit now). Defaults must be non-spammy. Notification bell
   island with unread count.
4. Worker process separate from the API process, graceful shutdown, retry with backoff, dead-letter
   list visible to admin.
TESTS: the verified flag cannot be forged by any payload; alerts fire once per price change.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): wishlist-এ বই রেখে দাম বদলান: ঠিক ১টি in-app + ১টি Mailpit ইমেইল আসে।

P11 Social feed ও moderation

D · Retention

কমিউনিটির ভিত্তি।

```
TASK: Community layer.

1. Posts (types: TEXT, IMAGE, REVIEW_SHARE, READING_UPDATE, QUOTE, QUESTION, ANNOUNCEMENT,
   SPONSORED), comments with one reply level, reactions (LIKE|DISLIKE exclusive), save, report,
   edit/soft-delete, hasSpoiler on posts/comments/reviews. Denormalised counters are updated in
   the same transaction as the action.
2. Feed API with cursor pagination (createdAt + id): following feed, trending feed (Redis cache
```

```
   60s, built by a job) and a mixed feed. Sponsored posts interleaved at most 1 per 6 items,
   always labelled "Sponsored", with a per-user frequency cap (feature flag FEATURE_SPONSORED).
   Publisher promotional posts capped at 3/day.
3. Moderation: Report queue, hide/remove/restore, user warn/suspend, ModerationLog, rate limits and
   a basic spam score (duplicate text, link flooding, mass actions) that flags but never auto-bans.
4. Web: feed island with infinite scroll (IntersectionObserver), skeletons, optimistic like with
   rollback, post composer with image upload, spoiler toggle, mention of books.
TESTS: feed order stable under concurrent inserts; blocked users' posts hidden; counters stay
correct under concurrent likes.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): feed p95 < 200ms (১০k পোস্ট seed)। N+1 কোয়েরি নেই (রিপোর্টে কোয়েরি সংখ্যা)।

P12 Bookshelf, goals, points, badges

D · Retention

PointLedger ছাড়া কেউ pointsBalance ছুঁতে পারবে না।

```
TASK: Reading identity and points.

1. Bookshelf (WANT_TO_READ, CURRENTLY_READING, READ, PAUSED) with page progress; moving a book
   to READ updates the yearly ReadingGoal. A reading update can optionally be shared as a post.
2. awardPoints(userId, reason, refId, delta) runs in a transaction that inserts a PointLedger row
   (unique userId+reason+refId prevents duplicates) and updates pointsBalance. No other code may
   touch pointsBalance.
3. Rules (all in SystemSetting): points for a DELIVERED purchase (not on order creation), first
   review of a book with minimum length, finishing a book, goal completion, helpful votes (daily
   cap). No points for likes/comments spam. Redeem: 100 points = 10 BDT, max 10% of an order;
   redemption is itself a ledger row and a cost the margin guard must account for.
4. Badges (Bookworm, Top Reviewer, Genre Explorer, Early Reader, Community Star) computed by a
   nightly job from ledger/review data; UserBadge unique per type.
5. Web: shelf board (move buttons first, drag-and-drop optional), progress ring, goal card, points
   history page.
TESTS: concurrent awards never double-count; balance always equals SUM(PointLedger.delta).
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): একটি স্ক্রিপ্ট প্রমাণ করে সব seed ইউজারের pointsBalance === SUM(delta)।

P13 Used-book marketplace ও ISBN scanner

E · Differentiators

FEATURE_USED_BOOKS=true করুন।

```
TASK: Peer-to-peer used books (enable FEATURE_USED_BOOKS).

1. Any verified user can create USED listings: book (search or ISBN), condition grade (LIKE_NEW,
   GOOD, FAIR, READABLE), at least 2 real photos, price, description, district/area, shipping or
   pickup. Admin moderation queue for new sellers' first listings.
2. ISBN scanner island (html5-qrcode, loaded only on that page): scan → GET /isbn/:isbn on our API
   → Open Library, then Google Books as fallback (check current usage terms/limits; cache
   results in DB). Show the candidate; the seller must CONFIRM before anything is created. Manual
   entry stays available.
3. Used books flow through the same checkout, escrow, claim and commission rules as new books
   (10% rule from Appendix D); stock is always 1; shipping through the courier flow.
4. "Near me": filter by district/area first. PostGIS radius search only if real demand appears.
5. Trust: seller rating after delivered orders, report listing. Duplicate-photo detection is out
   of scope.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): বারকোড স্ক্যান করে লিস্টিং ৩০ সেকেন্ডের মধ্যে। used-book অর্ডার টেস্টে পুরো money lifecycle চালায়।

P14 Mood discovery ও Recommendation E · Differentiators

LLM ছাড়াই চলে। রঙিন mood chip-গুলো এখানে কাজে লাগে।

```
TASK: Discovery. LOCAL-FIRST, LLM_PROVIDER=off by default.

1. Mood picker: 10 moods as soft coloured chips (use the MoodChip colours from Prompt 2). Phase 1
   is a plain DB query on Book.moodTags combined with price/length filters and popularity; no
   LLM. Copy must say "these books may match your mood" and never claim to diagnose emotion.
   Store MoodLog only with consent.
2. Recommendation service (rule-based, weights in SystemSetting): genre/author/publisher affinity
   from shelves, wishlist, purchases, follows; popularity; freshness; diversity so the list is not
   five books from one author. Sections: "Recommended for you", "Because you read X", "Trending in
   your genres", "Readers who follow X also follow Y" (plain SQL). Cache per user for 10 minutes.
3. Optional free-text box behind FEATURE_MOOD_LLM: LLM classification ONLY (text → one mood enum,
   strict JSON validated with Zod) through an LlmProvider interface, model name from env,
   per-user rate limit, Redis cache, rule-based fallback if the call fails. The LLM never invents
   books; results always come from our DB and only in-stock listings are shown.
4. Web: mood picker island on the homepage, results page, "why this book" tooltip.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): LLM বন্ধ রেখেও পুরো ফিচার চলে। স্টকহীন বই কখনো আসে না। cache-করা recommendation p95 < 150ms।

P15 Publisher growth tools E · Differentiators

FEATURE_SPONSORED=true।

```
TASK: Publisher tools.

1. Sponsored campaigns: budget, dates, targeting by genre and follower relationship only (no
   invasive profiling), impression/click/add-to-cart/purchase attribution via signed tracking
   ids, dashboard with CTR, conversion, revenue, cost, ROAS. Billing for now: manual invoice.
2. Reprint demand: "Request reprint" on out-of-stock books (one per user); publisher sees counts
   by date and district and a trend.
3. Release countdown with notify-me, pre-order flag, flash sale and bundle (buy 2 get 1) rules
   computed server-side in the quote service; the countdown UI uses server time.
4. Feature a standout user review/post (pin) with a notification to its author.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): analytics সংখ্যা seed করা raw event-এর সাথে মেলে। bundle/flash দাম ক্লায়েন্ট থেকে জাল করা যায় না।

P16 Writer tools, Events, Quote card E · Differentiators

FEATURE_LIVE_EVENTS=true। ভিডিও নিজে বানানো নয়।

```
TASK: Writer and live features.

1. Writer profile, blog posts, book announcements, follower-only content flag.
2. Beta-reader program: publisher/writer invites by eligibility score (verified purchases, review
   helpfulness, badges, low abuse score; not review count alone); invitation, deadline, required
   review; reward points through PointLedger.
3. AMA / live launch room with Socket.io (Redis adapter): moderated chat, pinned book with Buy
   button, question queue, event countdown, rate limits. No custom video streaming: store an
   external live link only.
4. Quote card generator: client-side html-to-image in a lazy island (the browser shapes Bangla
   conjuncts correctly; do NOT use server-side Satori without proving Bangla rendering). Formats
   1080x1920 story and 1080x1080 square with title, author, platform watermark and a short
   user-selected quote only.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): চ্যাট reconnect-এ টেকে। banned ইউজার রুমে লিখতে পারে না। বাংলা quote card-এ যুক্তাক্ষর ঠিক আসে (Android Chrome ও iOS Safari)।

P17 Local hardening ও লঞ্চ-প্রস্তুতি

F · Hardening

কোনো নতুন ফিচার নয়। লাইভ হওয়ার আগের পরীক্ষা।

```
TASK: Production readiness, local version. Do not add features.

1. Security review against OWASP Top 10: dependency audit, headers, CORS, CSRF/Origin, file upload
   abuse, SSRF in ISBN/image fetchers, raw SQL, mass assignment, IDOR sweep over EVERY route,
   rate-limit coverage report, admin 2FA.
2. k6 load test on the local machine: browse/search/checkout mix; report p95, error rate, DB
   connections and Redis memory (numbers are relative, not production capacity).
3. Reliability: `pnpm backup` (pg_dump to ./backups) AND a documented, tested restore; Redis
   persistence; BullMQ dead-letter visible in admin; structured logs with request ids.
4. Reconciliation: run `pnpm reconcile` and the points reconciliation on seeded + simulated data;
   review every ledger write path once more.
5. Terms, Privacy, Refund/Return and Seller Agreement drafts (Bangla + English) as static pages,
   unpublished until hosting exists.
6. Update docs/FUTURE_PLAN.md: every VERIFY-BEFORE-LAUNCH item with its status.
7. Prepare (do not run) Dockerfiles for web, api and worker so hosting later is a short step.
```

পাস হলে তবেই পরেরটা (ACCEPTANCE): সব টেস্ট সবুজ। backup→restore সফল। লিখিত রিপোর্ট: VERIFY-BEFORE-LAUNCH তালিকা, k6 ফল, নিরাপত্তা চেকলিস্ট।

Appendix A — Core Prisma schema (P3-এ পর্যন্ত)

```
enum Role { USER WRITER PUBLISHER VENDOR ADMIN RIDER }   // RIDER: feature flag OFF
enum UserStatus { ACTIVE SUSPENDED BANNED }
enum ListingType { NEW USED }
enum BookCondition { NEW LIKE_NEW GOOD FAIR READABLE }
enum PaymentMethod { COD SSLCOMMERZ }
enum PaymentStatus { INITIATED PAID FAILED REFUNDED }
enum OrderStatus {
  PENDING CONFIRMED PROCESSING PACKED SHIPPED OUT_FOR_DELIVERY
  DELIVERED CANCELLED RETURN_REQUESTED RETURNED REFUNDED DISPUTED
}
enum LedgerKind { SALE COMMISSION PAYMENT_FEE DELIVERY_FEE COD_REMIT RTO_COST REFUND PAYOUT ADJUSTMENT }
enum LedgerStatus { PENDING AVAILABLE PAID REVERSED }   // PENDING = escrow

model User {
  id              String     @id @default(cuid())
  email           String     @unique
  phone           String?    @unique
  phoneVerifiedAt DateTime?
  name            String
  avatarUrl       String?
  bio             String?
  role            Role       @default(USER)
  status          UserStatus @default(ACTIVE)
  isVerified      Boolean    @default(false)   // blue tick
  trustScore      Int        @default(50)      // COD risk, 0-100
  pointsBalance   Int        @default(0)       // cache; source of truth is PointLedger
  createdAt       DateTime   @default(now())
  updatedAt       DateTime   @updatedAt

  following   Follow[]  @relation("Follower")
  followers   Follow[]  @relation("Following")
  listings    Listing[]
  orders      Order[]   @relation("Buyer")
  ledger      LedgerEntry[] @relation("LedgerOwner")
  payouts     Payout[]
  checkouts   Checkout[]
  pointLedger PointLedger[]

  @@index([role, status])
}

model Follow {
  id          String   @id @default(cuid())
  followerId  String
  followingId String
  createdAt   DateTime @default(now())
  follower    User @relation("Follower", fields: [followerId], references: [id], onDelete: Cascade)
  following   User @relation("Following", fields: [followingId], references: [id], onDelete: Cascade)
  @@unique([followerId, followingId])
  @@index([followingId, createdAt])
}

model Book {
  id          String   @id @default(cuid())
  slug        String   @unique
  isbn        String?  @unique
  title       String
  titleBn     String?
  author      String
  publisher   String?
  description String?
  coverUrl    String?
  language    String   @default("bn")
  pageCount   Int?
  moodTags    String[]
  genreTags   String[]
```

```
  listings    Listing[]
  createdAt   DateTime @default(now())
  @@index([title])
  @@index([author])
}

model Listing {
  id          String        @id @default(cuid())
  bookId      String
  sellerId    String
  type        ListingType
  condition   BookCondition @default(NEW)
  price       Decimal       @db.Decimal(12, 2)
  mrp         Decimal?      @db.Decimal(12, 2)
  stock       Int           @default(1)
  reservedQty Int           @default(0)       // temporary hold during checkout
  images      String[]
  isActive    Boolean       @default(true)
  book        Book @relation(fields: [bookId], references: [id])
  seller      User @relation(fields: [sellerId], references: [id])
  items       OrderItem[]
  @@index([bookId, isActive])
  @@index([sellerId, isActive])
}

model Checkout {              // one buyer checkout; exactly one Payment
  id             String   @id @default(cuid())
  buyerId        String
  total          Decimal  @db.Decimal(12, 2)
  idempotencyKey String   @unique      // prevents double orders on double click
  createdAt      DateTime @default(now())
  buyer   User    @relation(fields: [buyerId], references: [id])
  orders  Order[]
  payment Payment?
}

model Order {                 // one Order per seller inside a Checkout
  id              String        @id @default(cuid())
  orderNumber     String        @unique
  checkoutId      String
  buyerId         String
  status          OrderStatus   @default(PENDING)
  paymentMethod   PaymentMethod
  subtotal        Decimal       @db.Decimal(12, 2)
  deliveryCharge  Decimal       @db.Decimal(12, 2)
  discount        Decimal       @default(0) @db.Decimal(12, 2)
  total           Decimal       @db.Decimal(12, 2)
  address         Json
  deliveredAt     DateTime?
  createdAt       DateTime      @default(now())
  buyer     User     @relation("Buyer", fields: [buyerId], references: [id])
  checkout  Checkout @relation(fields: [checkoutId], references: [id])
  items     OrderItem[]
  shipments Shipment[]
  @@index([buyerId, status])
  @@index([status, createdAt])
}

model OrderItem {
  id               String  @id @default(cuid())
  orderId          String
  listingId        String
  sellerId         String
  quantity         Int
  unitPrice        Decimal @db.Decimal(12, 2)    // snapshot
  commissionBps    Int                          // snapshot (1000 = 10%)
  commissionAmount Decimal @db.Decimal(12, 2)
  sellerAmount     Decimal @db.Decimal(12, 2)
  order   Order   @relation(fields: [orderId], references: [id], onDelete: Cascade)
  listing Listing @relation(fields: [listingId], references: [id])
  @@index([orderId])
```

```
  @@index([sellerId])
}

model Payment {
  id         String        @id @default(cuid())
  checkoutId String        @unique
  method     PaymentMethod
  provider   String?       // sslcommerz | mock
  tranId     String?       @unique
  amount     Decimal       @db.Decimal(12, 2)
  status     PaymentStatus @default(INITIATED)
  paidAt     DateTime?
  checkout   Checkout @relation(fields: [checkoutId], references: [id])
  events     PaymentEvent[]
}

model PaymentEvent {          // webhook idempotency
  id        String   @id @default(cuid())
  paymentId String
  provider  String
  eventId   String           // provider val_id / event id
  payload   Json
  createdAt DateTime @default(now())
  payment   Payment @relation(fields: [paymentId], references: [id])
  @@unique([provider, eventId])
}

model Shipment {
  id            String  @id @default(cuid())
  orderId       String
  courier       String          // mock | steadfast | pathao | redx
  consignmentId String?
  trackingCode  String?
  status        String
  codAmount     Decimal @default(0) @db.Decimal(12, 2)
  codRemittedAt DateTime?       // when the courier remitted the COD cash
  order         Order @relation(fields: [orderId], references: [id])
  @@index([orderId])
  @@unique([courier, consignmentId])
}

model LedgerEntry {           // amount/kind/owner immutable; only status & availableAt change (escrow service)
  id             String       @id @default(cuid())
  ownerId        String?                      // seller; null = platform
  orderItemId    String?
  kind           LedgerKind
  amount         Decimal      @db.Decimal(12, 2) // signed: + income, - deduction
  status         LedgerStatus @default(PENDING)
  availableAt    DateTime?                    // end of escrow
  idempotencyKey String       @unique
  note           String?
  createdAt      DateTime     @default(now())
  owner User? @relation("LedgerOwner", fields: [ownerId], references: [id])
  @@index([ownerId, status, availableAt])
}

model Payout {
  id        String   @id @default(cuid())
  sellerId  String
  amount    Decimal  @db.Decimal(12, 2)
  method    String           // bank | bkash | nagad
  reference String?
  status    String   @default("PENDING")
  paidAt    DateTime?
  createdAt DateTime @default(now())
  seller    User @relation(fields: [sellerId], references: [id])
}

model PointLedger {           // append-only
  id           String   @id @default(cuid())
  userId       String
```

```
  delta        Int
  reason       String
  refId        String?
  balanceAfter Int
  createdAt    DateTime @default(now())
  user User @relation(fields: [userId], references: [id])
  @@unique([userId, reason, refId])   // same reason cannot award twice
  @@index([userId, createdAt])
}

model CommissionRule {
  id        String   @id @default(cuid())
  scope     String            // GLOBAL | ROLE | SELLER | CATEGORY | LISTING_TYPE
  scopeRef  String?
  bps       Int
  priority  Int      @default(0)
  validFrom DateTime @default(now())
  validTo   DateTime?
}

model SystemSetting { key String @id  value Json  updatedAt DateTime @updatedAt }

model AdminAuditLog {
  id String @id @default(cuid())
  actorId String
  action String
  entity String
  before Json?
  after Json?
  createdAt DateTime @default(now())
  @@index([entity, createdAt])
}
```

Appendix B — Follow policy (P4-এ পরিষ্কৃত)

```
FOLLOW POLICY (enforced server-side; UI hiding is never security)

Matrix (actor → target):
  USER      → USER: yes | WRITER: yes | PUBLISHER: yes | VENDOR: yes
  WRITER    → USER: ONLY if that user already follows this writer | WRITER: yes | PUBLISHER: yes | VENDOR: no
  PUBLISHER → USER: NEVER | WRITER: yes | PUBLISHER: yes | VENDOR: no
  VENDOR    → nobody
  Nobody can follow themselves, ADMIN or RIDER accounts, or anyone they are blocked with.

Extra rules:
  1. Writer→User follow-back: the check and the insert happen in ONE DB transaction (race safe).
  2. When a user unfollows a writer, the writer's follow-back is removed (invariant: a writer only
     follows users who follow them).
  3. Blocking removes follows in both directions and prevents new ones.
  4. Follow rate limit: 50 per hour per user; mass-follow raises a risk score, it never auto-bans.
  5. Publisher promotional posts: max 3 per day. Sponsored posts always show a "Sponsored" label
     (Post.isSponsored).

Reference implementation shape:

export function canFollow(
  actor: { id: string; role: Role },
  target: { id: string; role: Role },
  ctx: { actorIsFollowedByTarget: boolean; blocked: boolean }
): { allow: true } | { allow: false; code: string } {
  if (actor.id === target.id || ctx.blocked) return deny('NOT_ALLOWED');
  if (target.role === 'ADMIN' || target.role === 'RIDER') return deny('NOT_ALLOWED');
  switch (actor.role) {
    case 'USER': return allow();
    case 'WRITER':
      if (target.role === 'USER') return ctx.actorIsFollowedByTarget ? allow() : deny('FOLLOW_BACK_ONLY');
      return ['WRITER', 'PUBLISHER'].includes(target.role) ? allow() : deny('NOT_ALLOWED');
    case 'PUBLISHER':
      return ['WRITER', 'PUBLISHER'].includes(target.role) ? allow() : deny('NOT_ALLOWED');
    default: return deny('NOT_ALLOWED');
  }
}
```

Appendix C — Money flow ৩ settings (P6, P7-এ পেস্ট)

```
MONEY FLOW (Checkout → one Order per seller; one Payment per Checkout)

SystemSetting defaults (never hard-code):
  claim_window_hours   48     buyer may open a claim this long after delivery
  escrow_days          7      seller money becomes AVAILABLE this many days after delivery
  min_payout_amount    500    BDT
  cod_new_user_cap     1500   BDT, max COD order for new users
  cod_min_trust        30     below this trustScore → prepaid only

PREPAID (SSLCommerz sandbox / mock):
  1. POST /checkout: ignore client prices; recompute price, stock, coupon, commission, delivery;
     Idempotency-Key header; reserve stock; Orders = PENDING; Payment = INITIATED.
  2. Create the provider session and return the gateway URL.
  3. On IPN: never trust redirects. Call the provider Validation API with val_id; status, tran_id,
     amount and currency must match our DB exactly. In ONE transaction: insert PaymentEvent
     (provider+eventId unique, duplicates stop here) → Payment PAID → Orders CONFIRMED → per
     OrderItem ledger entries: seller SALE (+, PENDING), platform COMMISSION, PAYMENT_FEE.
  4. Courier says DELIVERED: Order DELIVERED, deliveredAt set, seller entries get
     availableAt = deliveredAt + escrow_days.
  5. Hourly job: PENDING and availableAt ⩽ now and no open claim → AVAILABLE.
  6. Claim inside claim_window_hours: Order DISPUTED, entries frozen. Admin decides: refund writes
     negative REFUND entries (reverses seller and commission) or rejects; job resumes.
  7. Payout: weekly admin batch; AVAILABLE balance ⩾ min_payout_amount; Payout PENDING → PAID with
     a reference; entries become PAID.

COD:
  1. Order CONFIRMED without a gateway; Shipment.codAmount = order total.
  2. No ledger entries at order time. When the courier remits the cash (codRemittedAt) write seller
     SALE and commission; availableAt = max(deliveredAt, codRemittedAt) + escrow_days.
  3. RTO (returned parcel): restock, RTO_COST entry (who pays is config; default platform), buyer
     trustScore decreases.
  4. COD guards: phone verified; new-user cap; trustScore threshold; COD disabled after 3 RTOs.

Rules (tests required before merge):
  - Money in integer paisa, stored as Decimal(12,2); commission rounds half-up;
    sellerAmount + commissionAmount ≡≡ lineTotal exactly.
  - Every ledger write has an idempotencyKey (e.g. sale:{orderItemId}); retries never double-write.
  - LedgerEntry amount/kind/owner never change; only status and availableAt (escrow service).
    Corrections are new ADJUSTMENT entries.
  - Daily reconciliation: per paid order the ledger sums equal the order subtotal; mismatch alerts.
  - A seller can only read their own ledger. Every admin payout/refund/commission change is written
    to AdminAuditLog.
  - Margin guard: commissionAmount must never fall below estimated gateway fee + minimum margin.
  - Local test clock: an env-controlled "now" so escrow can be fast-forwarded.
```

Appendix D — Commission rules (P3, P6-এ ব্যবহৃত)

```
COMMISSION (Standard 10%; stored as CommissionRule rows, never constants)

Resolution: the most specific rule that is valid now wins; ties by higher priority.
Amounts are computed at checkout and snapshotted on OrderItem (commissionBps, commissionAmount, sellerAmount).

Seed rules:
  scope      scopeRef            bps    validTo          note
  GLOBAL     -                   1000   -                Standard 10% baseline for everyone
  ROLE       PUBLISHER_FOUNDING   400   +6 months        first 10 Founding Publishers (2-4% range)
  ROLE       PUBLISHER_VERIFIED   600   +6 months        launch offer
  ROLE       WRITER               600   +6 months        launch offer
  ROLE       VENDOR               800   +6 months        launch offer
  LISTING    USED                1000   -                used-book sales by users

After validTo the GLOBAL 10% applies automatically. Margin guard: show an admin warning whenever
a resolved commission is below estimated gateway fee + courier cost + minimum margin.
Every rule change writes AdminAuditLog (before/after).
```

মনে রাখবেন: কমিশন ১০% সাধারণ ভিত্তি। Founding অফারের ২–৪% শুধু প্রথম ১০ জন প্রকাশকের জন্য ৬ মাস। এর নিচে গেলে gateway ও courier খরচ উঠবে না। হার চূড়ান্ত হবে প্রকাশকদের সাথে আলোচনায়।